

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS2DA: Data Structures & Algorithms

Graphs and Finite State Automata

Recap

Augmenting trees:

- In BSTs, **minimum**, **maximum** and **range** are easy.
- Add size information to get fast **size**, **select** and **rank**.

With heaps we can implement:

- **Priority queue** with $\mathcal{O}(\log n)$ insert and delete minimum.
- **Heapsort**: In-place sort with guaranteed $\mathcal{O}(n \log n)$ time and $\mathcal{O}(1)$ space.
- **Treaps**: Map with average $\mathcal{O}(\log n)$ insert and delete.

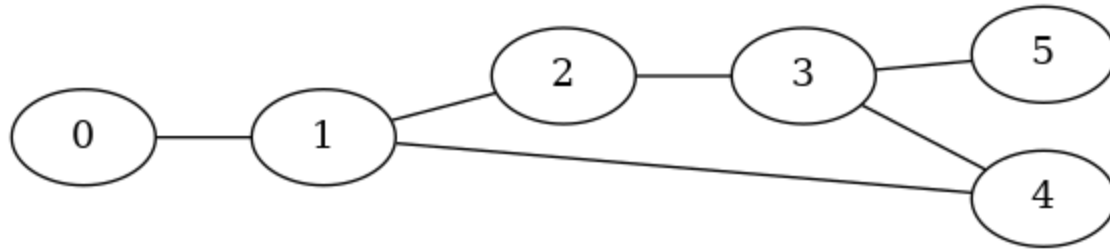
Min-heap property:

- Every node is smaller than its children.

Today

- Graphs
 - Undirected and directed graphs
 - Adjacency matrix vs adjacency list representation
- Finite State Automata (FSA)
 - Deterministic Finite State Automata (DFA)
 - Nondeterministic Finite State Automata (NFA)
- Regular Expressions (regexes)
 - Thompson's construction — convert regexes to NFA using D&C
- Looking ahead: FSA algorithms using techniques later in module
 - Powerset construction — convert NFA to DFA with BFS/DFS
 - Knuth-Morris-Pratt — efficient NFA to DFA conversion with dynamic programming for fast substring matching
 - Lexical analysis — greedy algorithm for longest matching token

Undirected Graphs



Graphs are ubiquitous in computer science and its applications.

A **graph** G is a network of **nodes** or **vertices** V joined by **edges** E .

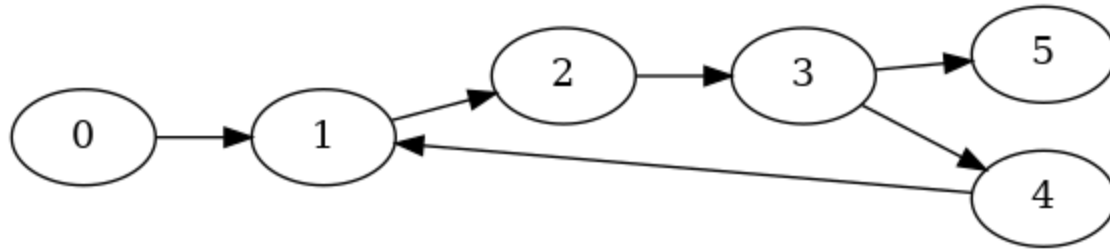
In an **undirected graph**, edges can be **traversed** in both directions.

A **path** is a sequence of nodes joined by edges.

Nodes joined by edges are **adjacent**.

Graph Variations

In a **directed** graph, edges can only be traversed in one direction.



Mathematicians usually first consider undirected graphs, but in Computer Science, the directed graph is arguably more fundamental. Undirected is a special case.

A **simple** graph forbids self-loops and parallel edges.

Graph Applications

application	node	edge
road map	town	road
website	page	link
circuit	electronic component	wire
computer network	server/router	Ethernet cable
software	method	call
social network	person	friendship
data structure	object	pointer

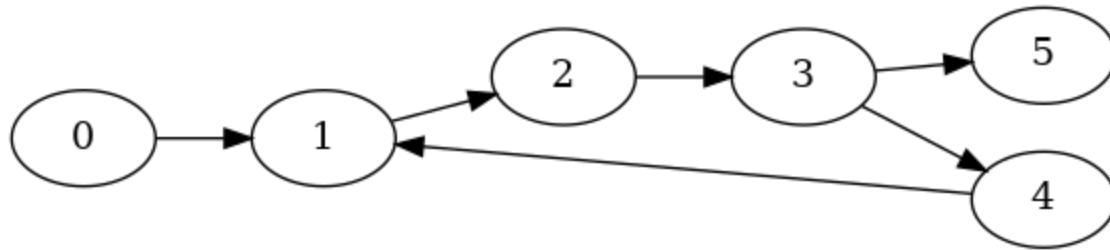
Adjacency Matrix Representation

Suppose a graph's nodes are numbered from 0 upwards.

An adjacency matrix uses a 2D array to store edges.

$A[i][j] = 1$ when there is an edge from node i to node j .

Example:



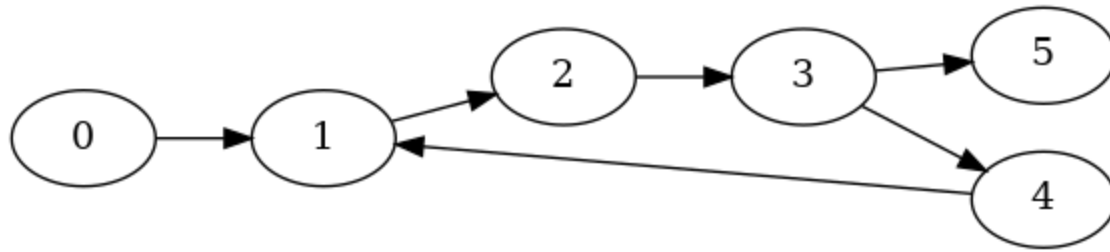
Edge lookup is $\mathcal{O}(1)$, but uses $\mathcal{O}(|V|^2)$ space. Good for **dense** graphs.

Adjacency List Representation

An adjacency list uses an array of linked lists to store edges.

$A[i]$ is a list of vertices adjacent to node i .

Example:



Only uses $\mathcal{O}(|E|)$ space, but edge lookup is $\mathcal{O}(|V|)$.

As E is bounded by V^2 , good for **sparse** graphs.

Adjacency List Variations

We can use hash tables or balanced binary trees instead of linked lists.

We can use hash tables or balanced binary trees instead of an array.

Particularly good if vertices are named (with strings) rather than numbered.

Use hash tables everywhere:

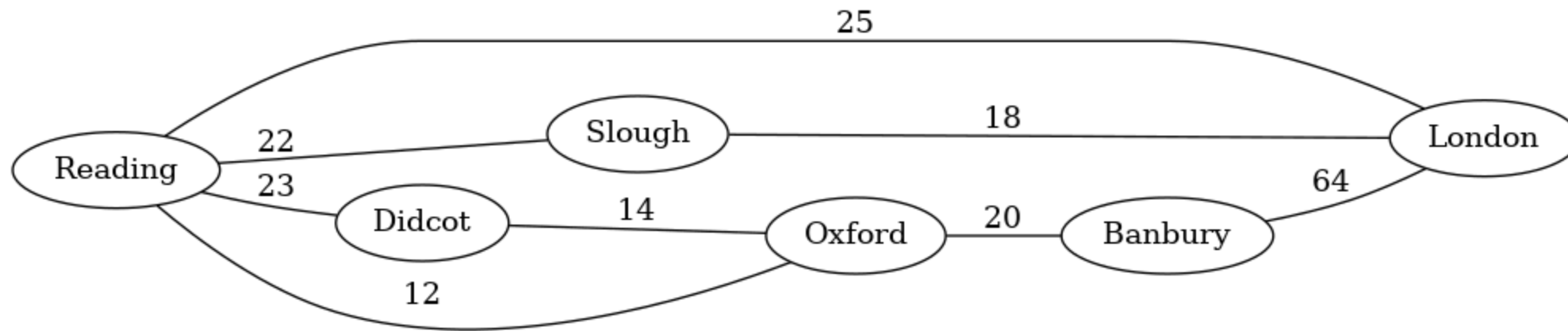
edge lookup is average $\mathcal{O}(1)$; space is $\mathcal{O}(|E|)$.

Use balanced binary trees everywhere:

edge lookup is guaranteed $\mathcal{O}(\log |V|)$; space is $\mathcal{O}(|E|)$.

Node Labels & Edge Labels

We often label nodes and edges in a graph.



We can add extra storage to the graph data structure to hold this information.

Substring Matching and Regular Expressions

Substring Matching

We often need to search text for particular keywords. Examples:

- Find and replace all occurrences of a variable name in source code.
- Search e-mail archive for references to a particular student.

Regular Expression Matching

Sometimes we need to find more than just keywords. Examples:

- Find all files with names like `image000.png` , `image932.png` .
- Look for words at the start of sentences with no capital letter.

We can solve these problems efficiently, but we need to develop some theory first...

(Deterministic) Finite State Automota

Automata

Automaton (plural: automata) is just another word for machine.

In Computer Science, we often use **automaton** to refer to a machine that has less computational power than a computer (such as a Turing Machine).

Finite State Automata

An FSA is a machine that recognises **words** (sequences of characters).

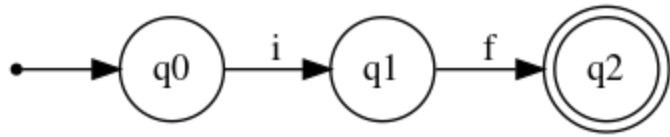
If you give it an input word, it either accepts it ("**yes**") or rejects it ("**no**").

The set of words a particular FSA recognises is called its **language**.

The class of languages accepted by FSA is called **regular languages**.

We can use FSA to recognise patterns (**substrings** or **regular expressions**) in text strings.

What is a Finite State Automaton?



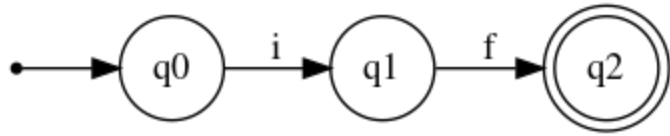
The "program" for an FSA is usually represented as a directed graph.

Nodes in the graph are **states**. The automaton can only be in one state at a time.

There is exactly one **initial state**, marked by an incoming arrow.

Some of the states are **accepting** or **final**. They are usually marked with a circle.

What is a Finite State Automaton?

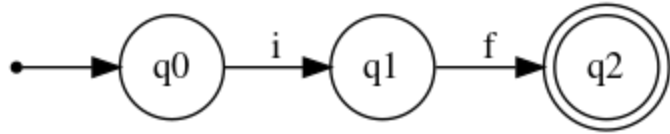


Edges in the graph are **transitions**. They indicate which state the automaton can move to next.

Labels on edges tell the automaton which transitions it can take when it sees a certain character in its input.

FSA read their input word one character at a time, in sequence. They cannot rewind their input or save it.

What is a Finite State Automaton?



If you are not sure what an automaton does, try tracing its behaviour on some examples:

Word read	End state	Accept?
ϵ	q_0	X
i	q_1	X
if	q_2	✓

(ϵ denotes the **empty word**.)

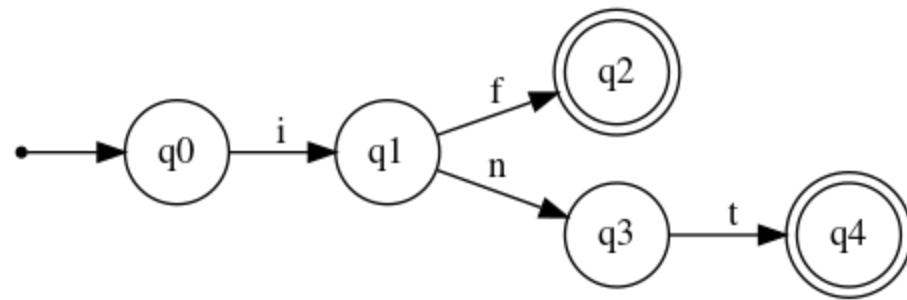
This automaton accepts the word **if** and nothing else.

It recognises this language: { **if** }

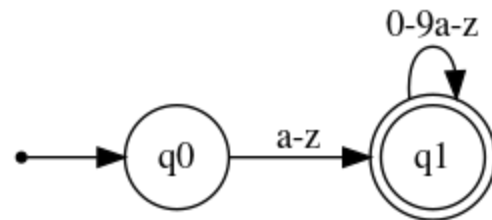
Examples

Exercise: What languages do these automata accept?

1.

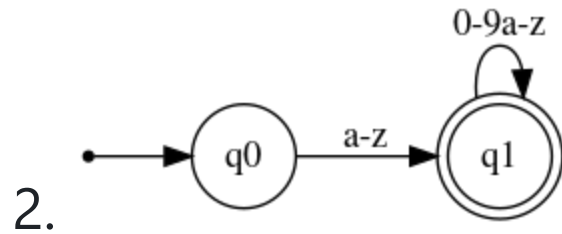
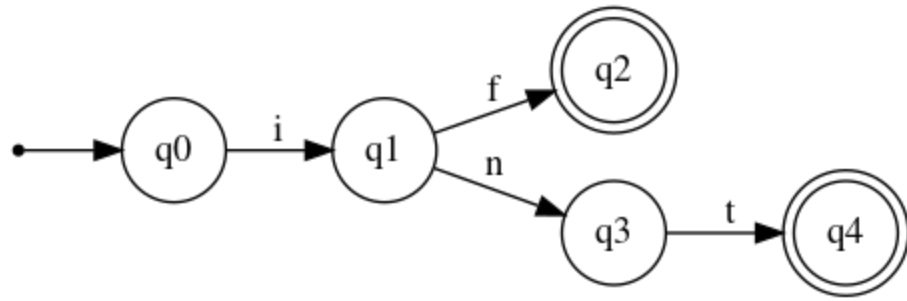


2.



Examples

Exercise: What languages do these automata accept?



1. { `if`, `int` }

2. Alphanumeric strings beginning with a letter.

Notation used for FSA

If you look at textbooks on FSA, you will probably see this notation:

- Σ - alphabet (set of possible input characters)
- Q - set of all states
- q_0 - initial state
- F - set of accepting/final states
- δ or $\rightarrow$ - transition function or transition relation
- A - an automaton
- $\mathcal{L}(A)$ - language accepted by automaton A

A **run** of an FSA is the sequence of states (path) it goes through while reading a word:

$$q_0 \xrightarrow{w} q_1 \xrightarrow{h} q_2 \xrightarrow{i} q_3 \xrightarrow{l} q_4 \xrightarrow{e} q_5$$

Examples

Exercise: Draw FSA that accept the following languages over the alphabet $\{0, 1\}$:

- Words containing no 0s.
- Words containing no 0s and at least one 1.

Examples

Exercise: Draw FSA that accept the following languages over the alphabet $\{0, 1\}$:

- Words containing at most two 0s.
- Words containing an odd number of 1s.

Examples

Exercise: Draw FSA that accept the following languages over the alphabet $\{0, 1\}$:

- Words containing at most two 0s, or an odd number of 1s.

Nondeterministic Finite State Automata

Nondeterminism

What happens if an FSA has multiple transitions out of a state on the same character?

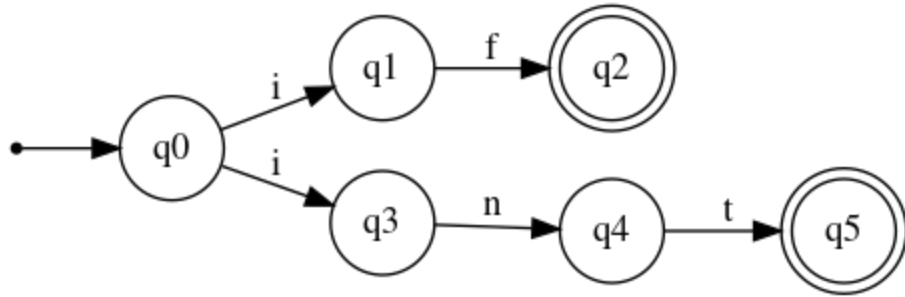
Recall:

An FSA **accepts** a word if there is a path from the start state to an accepting state where the labels on the edges spell the word.

An FSA is **nondeterministic** if there is sometimes more than one choice.

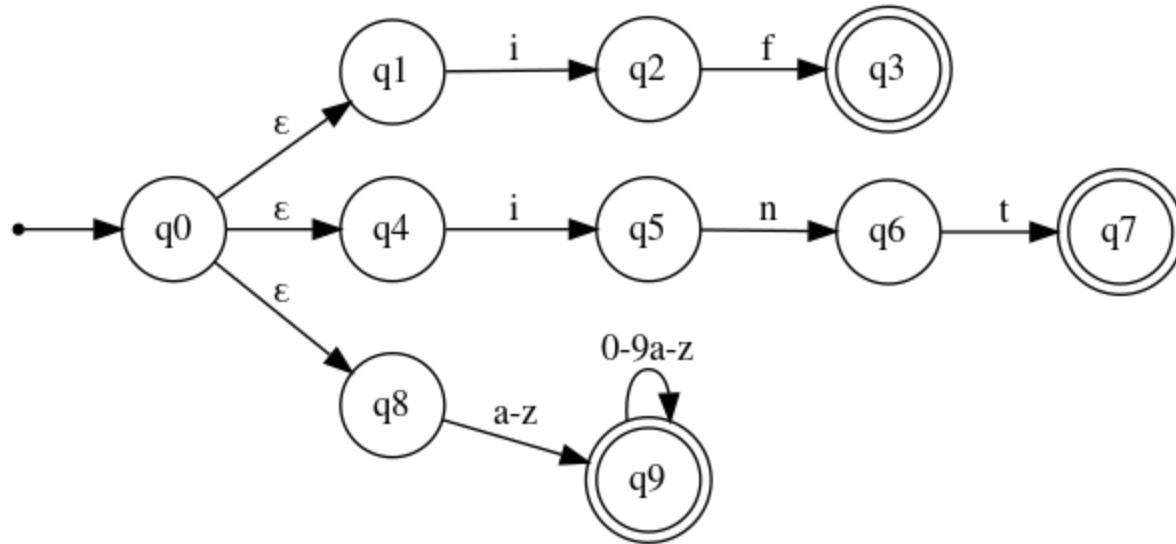
Using nondeterminism sometimes makes it easier to design automata matching a certain language.

Nondeterminism



Word read	End states	Accept?
ϵ	q_0	X
i	q_1, q_3	X
if	q_2	✓
ift	-	X
in	q_4	X
int	q_5	✓

Epsilon transitions



For convenience, we also use a special label ε to indicate a transition that can be taken without consuming an input character.

Caution: The nondeterministic transition ε is not the same as the empty word ϵ .

Examples

Exercise: Draw an NFA that accepts the following language over alphabet $\{0, 1\}$:

- Words containing at most two 0s, or an odd number of 1s.

Implementing deterministic FSA

An FSA is a **deterministic finite automaton (DFA)** if it has no nondeterminism.

It is easy to write a program that implements a DFA (in pseudocode):

```
int state = q0
while ((c = getchar()) != EOF)
    state = transition(state, c)

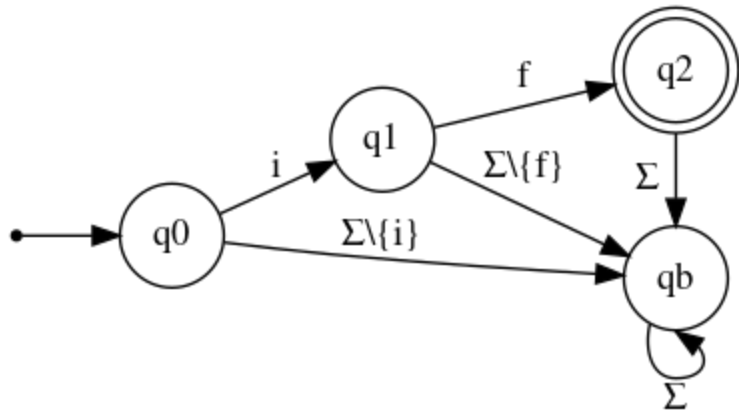
return accepting(state)
```

The function `transition` does a lookup in the underlying graph's adjacency matrix/list.

Implementing deterministic FSA

Some texts say deterministic automata must be **complete**: every state has **exactly** one transition for each character, not at most one.

We can easily make a DFA complete by adding a **bottom** state.



Some transformations are easier on complete DFA.

Implementing nondeterministic FSA

An FSA is a **nondeterministic finite automaton (NFA)** if it has some nondeterminism.
How can we implement it?

1. Use backtracking

- Whenever there are multiple choices for the next state, pick one.
- If we get to the end of a word and it is not accepted, go back and pick another.
- We can only reject after trying all combinations of choices.
- Fairly easy to implement, but may be very slow to run.

Implementing nondeterministic FSA

An FSA is a **nondeterministic finite automaton (NFA)** if it has some nondeterminism.
How can we implement it?

2. Evaluate all options in parallel

- The automaton now has a set of current states, not just one.
- If we get to the end of a word and any of the current states is accepting, the automaton accepts the word.
- Some people draw analogies with quantum physics.
- Harder to implement, but good performance.

3. Convert the NFA to a DFA...

Parallel Evaluation

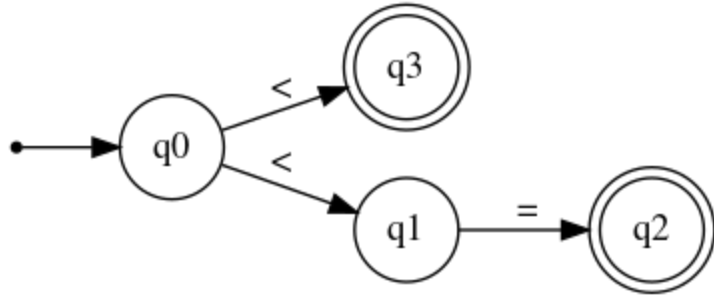
```
set states = eps(q0)
while ((c = getchar()) != EOF)
    set next_states = empty
    for each q1 in states
        for each q2 in transition(q1, c)
            next_states = next_states union eps(q2)
    states = next_states

for each q in states
    if accepting(q)
        return true

return false
```

where `eps(q)` returns the set of states reachable from `q` with only epsilon transitions.
Time complexity is $\mathcal{O}(n|Q|^2)$.

Converting NFA to DFA



Exercise: Find a DFA equivalent to this NFA.

We can always convert NFA to DFA using the **powerset construction**:

- The DFA may be exponentially larger than the NFA.
- DFA fast to run, but may be large and slow to compile.
- As with parallel evaluation, have to handle ϵ transitions specially.
- If NFA has states Q , DFA has states $\mathcal{P}(Q)$.
- Initial state of DFA is $\{eps(q_0)\}$.

Regular expressions

It looks like it might be easy to write a program that implements an FSA to recognise a particular language. But:

- It is not always easy to design an FSA for a language.
- It is not convenient to write a directed graph in a text editor.

Fortunately, there is an equivalent formalism without these disadvantages: **regular expressions** (sometimes abbreviated to **regexes**).

Caution: Some programming languages, such as Perl, use **regex** to refer to a syntactically similar but more powerful formalism.

Syntax of regular expressions

Base regular expressions:

- $\emptyset$ - empty language
- ϵ - empty string
- a - single characters

Compound regular expressions (where R_1, R_2, R are regular expressions):

- $R_1 \mid R_2$ - alternation (either/or/both)
- $R_1 . R_2$ - concatenation (one after the other)
- R^* - Kleene star (zero or more repetitions)

Examples

Some simple examples of regular expressions:

- $a(a^*)h$ — ah, aah, aaah, ...
- $(a \mid the)(cat \mid dog)$ — a cat, a dog, the cat, the dog

What do the following regular expressions over the alphabet $\{0, 1\}$ mean?

- $11 \mid 0$
- $(1 \mid 0)^*$
- $(10)^*$
- 10^*
- $(1 \mid \epsilon).0^*$

Exercise: For each word of at most 2 characters, decide which languages contain it.

Translating regular expressions to FSA with D&C

Work recursively. We just need to handle each kind of regular expression.

The details are simplified if we ensure that the automata we generate:

- have exactly one initial and one accepting state;
- have no transitions into the initial state;
- have no transitions out of the accepting state.

Exercise: This is called **Thompson's construction**.

Work out the details or look them up.

The reverse translation, from FSA into regular expressions, is called **Kleene's algorithm**, and is more complex.

Limitations of FSA

FSA can recognise a useful, well-defined, but ultimately limited class of languages.

Intuitively, languages that require "counting" can't be recognised by FSA.

Example: The language $a^k b^k$, where k is a constant, can't be recognised.

(**Note:** Perl extended regexes can recognise this language.)

But we need to be able to count to match brackets for most computer programming languages.

So we need a different technique to parse programming languages.

Efficiency

For running an NFA over a string, both backtracking search and conversion to DFA with **powerset construction** have exponential worst-case complexity.

If a program accepts regexes from an untrusted user (example: Web search), it must use the parallel evaluation strategy or risk Denial of Service attacks.

For the special case of substring search, there is an efficient **dynamic programming** algorithm (**Knuth-Morris-Pratt**) to convert the NFA to a DFA.

A common application for DFA is **lexical analysis**: splitting program source code into **tokens**. An optimised representation of the DFA is needed to keep it small.

In lexical analysis, there might be multiple choices about how to split source code into tokens. The **greedy** algorithm for **longest match** is used to decide.

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

Next time: FIFO Queues, LIFO Stacks,
DFS/BFS Graph Traversal